PROJET AFRIGARI

RAPPORT DE CONCEPTION LOGICIELLE & DIAGRAMME DE CLASSES

Analyse, Correction et Validation de la Structure de Données

Destinataire : Jury de Mémoire de Stage
Auteur : Stagiaire Afrigari
Technologie : NestJS / Next.js / TypeScript
Date de génération : Juin 2026

1. Introduction et Contexte

Dans le cadre de la rédaction de notre mémoire de stage, ce rapport présente l'analyse, la correction et l'extension du diagramme de classes de l'application Afrigari. L'objectif de cette démarche est de garantir une cohérence parfaite entre la modélisation UML théorique et l'implémentation physique en production (code source NestJS et base de données multi-cibles).

Nous avons révisé l'ensemble des entités métiers, corrigé les incohérences structurelles, et ajouté les méthodes opérationnelles essentielles. Ce rapport valide également la structure au regard du code source réel du projet.

Note sur la notation de visibilité : Conformément aux règles UML personnalisées du projet, les attributs ne portent aucun signe de visibilité. Seules les méthodes possèdent des indicateurs de visibilité : '+' pour public, '-' pour privé, et '*' pour protégé.

2. Liste des Corrections Apportées au Diagramme

- Doublon de la classe Boutique :
Une classe Boutique vide était présente à gauche pour la relation de signalement. Elle a été fusionnée avec la classe Boutique principale à droite.

- Correction du type 'caractère' :
Le type 'caractère' (char) a été remplacé par 'chaine de caractères' (string) pour les attributs de noms (Utilisateur, Boutique, Produit) et de rôles.

- Type de numéro de téléphone :
Le numéro de téléphone de Boutique était un entier. Il a été modifié en 'chaine de caractères' pour conserver le 0 initial et supporter les indicatifs internationaux (ex: +225).

- Type de l'identifiant externe :
L'identifiant de transaction (id_externe) du Paiement a été typé en 'chaine de caractères' pour accepter les chaînes de hachage des passerelles (Stripe, etc.).

- Orthographe des statuts :
L'attribut 'statue' présent dans de nombreuses classes (commande, paiement, etc.) a été corrigé en 'statut'.

- Lien Commande-Produit :
Remplacement de l'attribut textuel 'articles' par une modélisation correcte sous forme d'attribut JSON compatible avec le stockage flexible simple-json de TypeORM.

- Corrections de fautes de frappe :
Correction de 'emai' en 'email', 'mot_passe' en 'mot_de_passe', et 'chaine de caratère' en 'chaine de caractères'.

- Mise à jour des visibilités :
Retrait des signes de visibilité (+) devant tous les attributs. Application stricte de la visibilité uniquement sur les méthodes (+ pour public, - pour privé, * pour protégé).

3. Dictionnaire des Classes, Attributs et Méthodes

Classe : Utilisateur

Attributs :

id	: chaine de caractères	- Identifiant unique (UUID)
nom	: chaine de caractères	- Nom complet
email	: chaine de caractères	- Adresse email (unique)
mot_de_passe	: chaine de caractères	- Mot de passe haché
role	: chaine de caractères	- Rôle (CLIENT, VENDEUR, ADMIN)
statut_verification	: chaine de caractères	- Statut du KYC vendeur
est_verifie	: booléen	- Vérification OTP email réalisée

Méthodes :

+ sInscrire()	-> booléen	- Inscrit un nouvel utilisateur (Public)
+ seConnecter()	-> chaine de caractères	- Authentifie et retourne le token JWT (Public)
+ seDeconnecter()	-> booléen	- Invalide la session actuelle (Public)
+ mettreAJourProfil()	-> booléen	- Met à jour les données du profil (Public)
+ soumettreKYC()	-> booléen	- Envoie les pièces d'identité vendeur (Public)
- verifierFormatEmail()	-> booléen	- Vérifie le format regex de l'email (Privé)
* genererTokenTemporaire()	-> chaine de caractères	- Génère un token de récupération (Protégé)

Classe : Boutique

Attributs :

id	: chaine de caractères	- Identifiant unique (UUID)
nom	: chaine de caractères	- Nom de la boutique
description	: chaine de caractères	- Description commerciale
adresse	: chaine de caractères	- Localisation physique
telephone	: chaine de caractères	- Contact téléphonique

Méthodes :

+ creerBoutique()	-> Boutique	- Initialise une nouvelle boutique (Public)
+ modifierInfos()	-> booléen	- Modifie les informations de la boutique (Public)
+ obtenirStatistiquesVentes()	-> JSON	- Fournit les données du tableau de bord (Public)
- validerCoordonneesGPS()	-> booléen	- Valide la latitude et la longitude (Privé)

Classe : Produit

Attributs :

id	: chaine de caractères	- Identifiant unique (UUID)
nom	: chaine de caractères	- Désignation du produit
prix	: réel	- Prix unitaire
stock	: entier	- Quantité disponible
unite	: chaine de caractères	- Conditionnement (ex: Sac, Kg, Pièce)

Méthodes :

+ creerProduit()	-> Produit	- Ajoute le produit au catalogue (Public)
+ modifierProduit()	-> booléen	- Modifie les caractéristiques du produit (Public)
+ mettreAJourStock(quantite)	-> void	- Ajuste le stock lors d'une vente/réapprovisionnement (Public)

Classe : commande

Attributs :

id	: chaine de caractères	- Identifiant unique (UUID)
montant_total	: réel	- Montant total facturé
statut	: chaine de caractères	- Statut (PENDING, DELIVERED, CANCELED)
montant_commission	: réel	- Frais de commission d'Afrigari
articles	: JSON	- Liste des articles commandés stockés en JSON

Méthodes :

+ creerCommande()	-> commande	- Enregistre une commande client (Public)
+ mettreAJourStatut(nouveauStatut)	-> booléen	- Met à jour l'avancement de la livraison (Public)
+ calculerMontantTotal()	-> réel	- Calcule le prix final de la commande (Public)
- appliquerTaxes()	-> réel	- Applique les taxes de vente en vigueur (Privé)
* calculerCommission()	-> réel	- Calcule la part de la commission pour Afrigari (Protégé)

3. Dictionnaire des Classes (Suite)

Classe : Paiement

Attributs :

id	: chaine de caractères	- Identifiant unique (UUID)
id_externe	: chaine de caractères	- Identifiant Stripe/Paypal
montant	: réel	- Montant débité
statut	: chaine de caractères	- Statut (PENDING, SUCCESS, FAILED)
methode_paiement	: chaine de caractères	- Moyen de paiement (CARTE, MOBILE_MONEY)

Méthodes :

+ initierPaiement()	-> Paiement	- Crée l'intention de paiement externe (Public)
+ confirmerPaiement()	-> booléen	- Valide le paiement après webhook (Public)
+ rembourser()	-> booléen	- Déclenche un remboursement de transaction (Public)

Classe : Signalement

Attributs :

id	: chaine de caractères	- Identifiant unique
motif	: chaine de caractères	- Type de signalement (fraude, abus)
statut	: chaine de caractères	- Statut (NEW, INVESTIGATING, RESOLVED)
description	: chaine de caractères	- Explication détaillée de l'abus

Méthodes :

+ soumettreSignalement()	-> Signalement	- Enregistre le rapport par un utilisateur (Public)
+ mettreAJourStatut()	-> booléen	- Modifie l'état d'investigation (Public)

Classe : DemandeSupport

Attributs :

id	: chaine de caractères	- Identifiant unique
sujet	: chaine de caractères	- Objet de la demande
statut	: chaine de caractères	- Statut du ticket (OPEN, CLOSED)
categorie	: chaine de caractères	- Catégorie technique ou facturation

Méthodes :

+ ouvrirTicket()	-> DemandeSupport	- Initialise un ticket de support (Public)
+ fermerTicket()	-> booléen	- Clôture la demande d'assistance (Public)

Classe : Discussion & Message

Attributs :

Discussion.dernier_message	: chaine de caractères	- Dernier message envoyé
Discussion.date_dernier	: date-heure	- Date du dernier échange
Message.texte	: chaine de caractères	- Contenu textuel du message
Message.est_lu	: booléen	- Indicateur de lecture

Méthodes :

+ creerDiscussion()	-> Discussion	- Crée une conversation client-vendeur (Public)
+ recupererMessages()	-> Liste<Message>	- Récupère l'historique de la discussion (Public)
+ envoyerMessage()	-> Message	- Enregistre et transmet un nouveau message (Public)

+ marquerCommeLu()	-> void	- Marque le message comme lu (Public)

4. Vérification de la Cohérence avec le Code Source

Une étape importante de notre stage a consisté à vérifier la parfaite correspondance entre ce diagramme de classes UML conceptuel et le code TypeScript implémenté dans le dossier backend/src/core. Voici les points de validation clés :

1. Modèle Commande (Order) :
L'implémentation de la classe 'Order' dans 'order.entity.ts' contient bien l'attribut 'items' typé en simple-json TypeORM. Ceci correspond précisément à l'attribut 'articles' de notre diagramme. Cette technique a été adoptée pour garantir la flexibilité et la compatibilité du stockage entre SQLite, PostgreSQL et MongoDB, en sérialisant les articles achetés dans la table.

2. Modèle Paiement (Paiement / Transaction) :
Dans le code source backend/src/core/payments/transaction.entity.ts, cette table est nommée 'Transaction' mais elle possède exactement la structure de notre classe 'Paiement' (id, orderId, externalId, amount, status, paymentMethod), validant ainsi notre modèle.

3. Support et Chat en Temps Réel :
La classe 'Discussion' et la classe 'Message' se retrouvent dans l'entité 'Conversation' et 'Message' du module de chat. Le support est également implémenté avec l'entité 'Ticket' et 'TicketMessage' (support/ticket.entity.ts), confirmant l'exactitude de nos classes de messagerie.

4. Extensions identifiées dans le Code source (Non présentes sur le diagramme initial) :
L'analyse du code source révèle deux entités supplémentaires pour le catalogue produit : - La classe 'Favorite' (favorites/favorite.entity.ts) : gère la relation many-to-many entre les clients et leurs produits favoris.
- La classe 'ProductReview' et 'Review' (products/product-review.entity.ts & reviews/review.entity.ts) : gèrent les avis laissés par les acheteurs sur les produits et sur l'application.

5. Conclusion

La révision et la correction du diagramme de classes ont permis d'éliminer les erreurs de modélisation théorique de notre infrastructure logicielle. L'ajout des méthodes pour chaque classe offre une vue claire du comportement fonctionnel de la plateforme.

La confrontation avec le code source réel (NestJS / TypeORM) démontre que les choix architecturaux majeurs, tels que le stockage simple-json pour les articles de la commande et l'abstraction multi-moteurs de bases de données, ont été fidèlement transcrits, assurant ainsi un travail de conception rigoureux et de qualité universitaire pour notre mémoire.